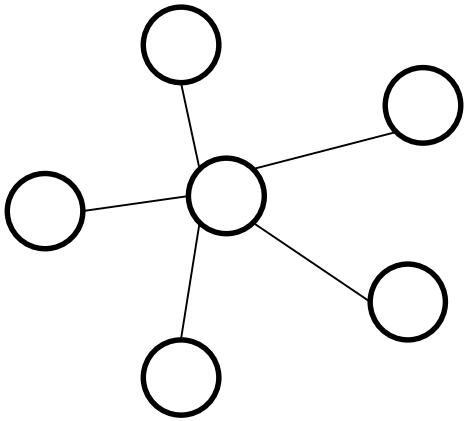
Introduction to Trees

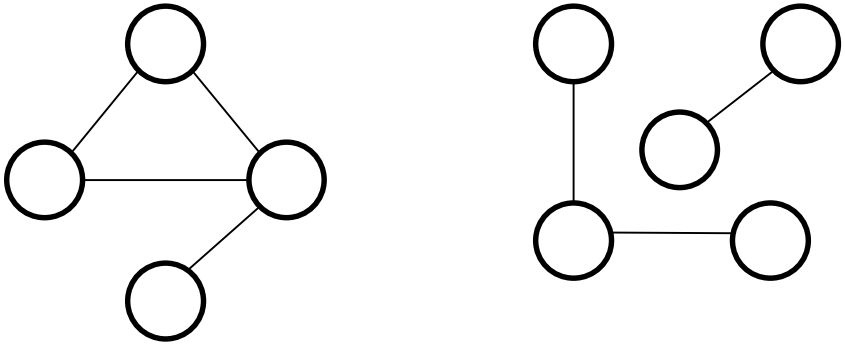
Tree

A tree is a graph with the following properties:

- The graph is connected (can go from anywhere to anywhere)
- There are no cycles(acyclic)



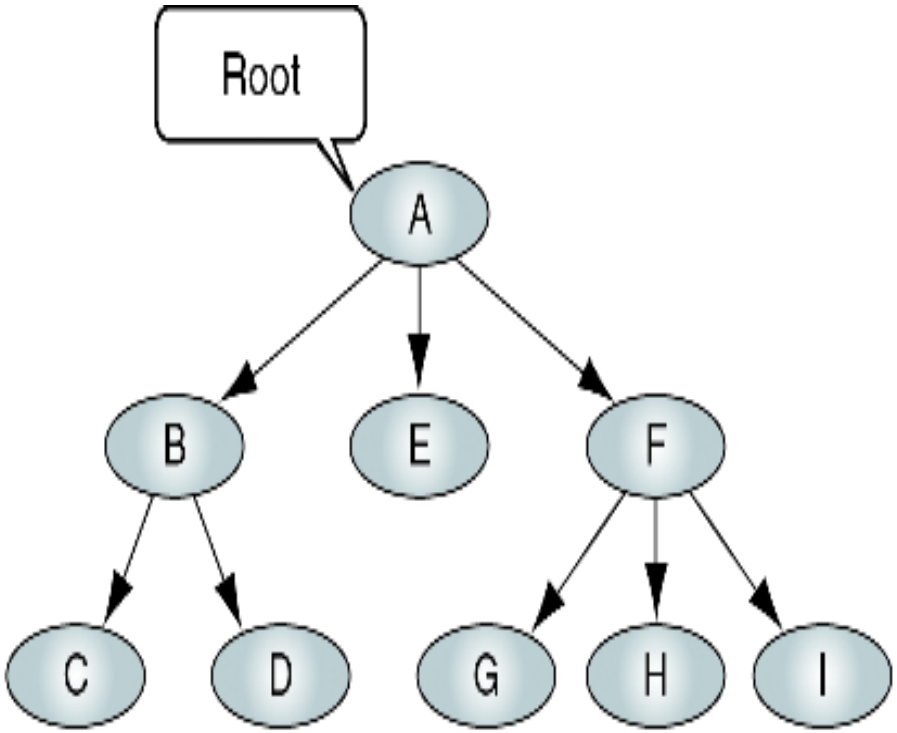
Tree



Graphs that are not trees

Definition

- A tree consists of finite set of elements, called nodes, and a finite set of directed lines called branches, that connect the nodes.
- A tree is a connected graph without any circuits



Tree

Basic Tree Concepts

- When the branch is directed toward the node, it is indegree branch.
- When the branch is directed away from the node, it is an outdegree branch.
- The sum of the indegree and outdegree branches is the degree of the node.
- If the tree is not empty, the first node is called the root.

Basic Tree Concepts

- The indegree of the root is, by definition, zero.
- With the exception of the root, all of the nodes in a tree must have an indegree of exactly one; that is, they may have only one predecessor.
- All nodes in the tree can have zero, one, or more branches leaving them; that is, they may have outdegree of zero, one, or more.

Basic Tree Concepts

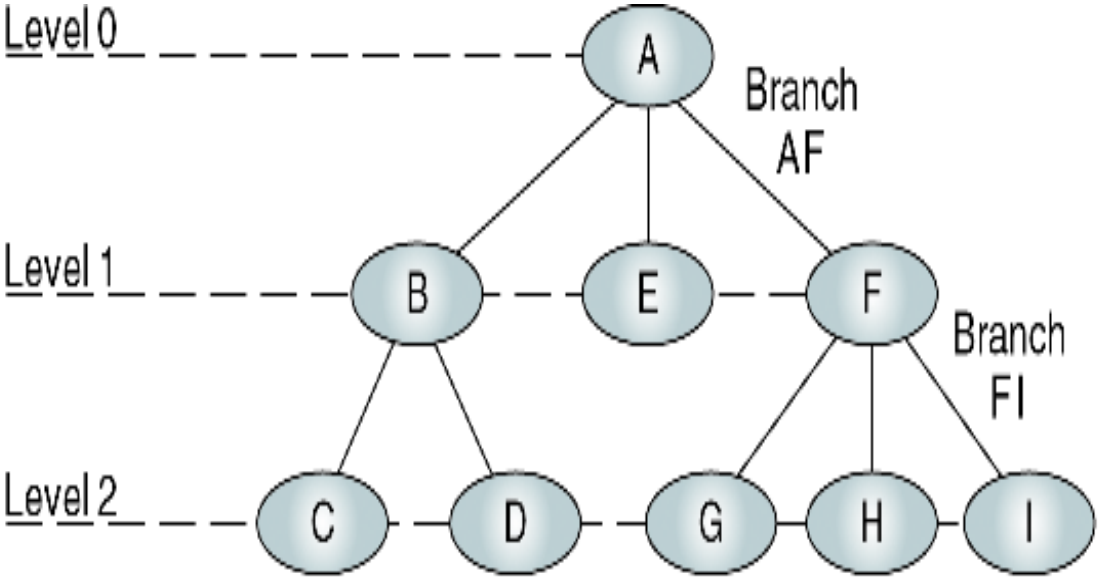
- A leaf is any node with an outdegree of zero, that is, a node with no successors.
- A node that is not a root or a leaf is known as an internal node.
- A node is a parent if it has successor nodes; that is, if it has outdegree greater than zero.
- A node with a predecessor is called a child.

Basic Tree Concepts

- Two or more nodes with the same parents are called siblings.
- A path is a sequence of nodes in which each node is adjacent to the next node.
- The level of a node is its distance from the root. The root is at level 0, its children are at level 1, etc. …

Basic Tree Concepts

- The height of the tree is the level of the leaf in the longest path from the root plus 1. By definition the height of any empty tree is 1.
- A subtree is any connected structure below the root. The first node in the subtree is known is the root of the subtree.



Root:	A	Siblings:	{B,E,F}, {C,D}, {G,H,I}
Parents:	A, B, F	Leaves:	C,D,E,G,H,I
Children:	B, E, F, C, D, G, H, I	Internal nodes:	B,F

FIGURE 6-2 Tree Nomenclature

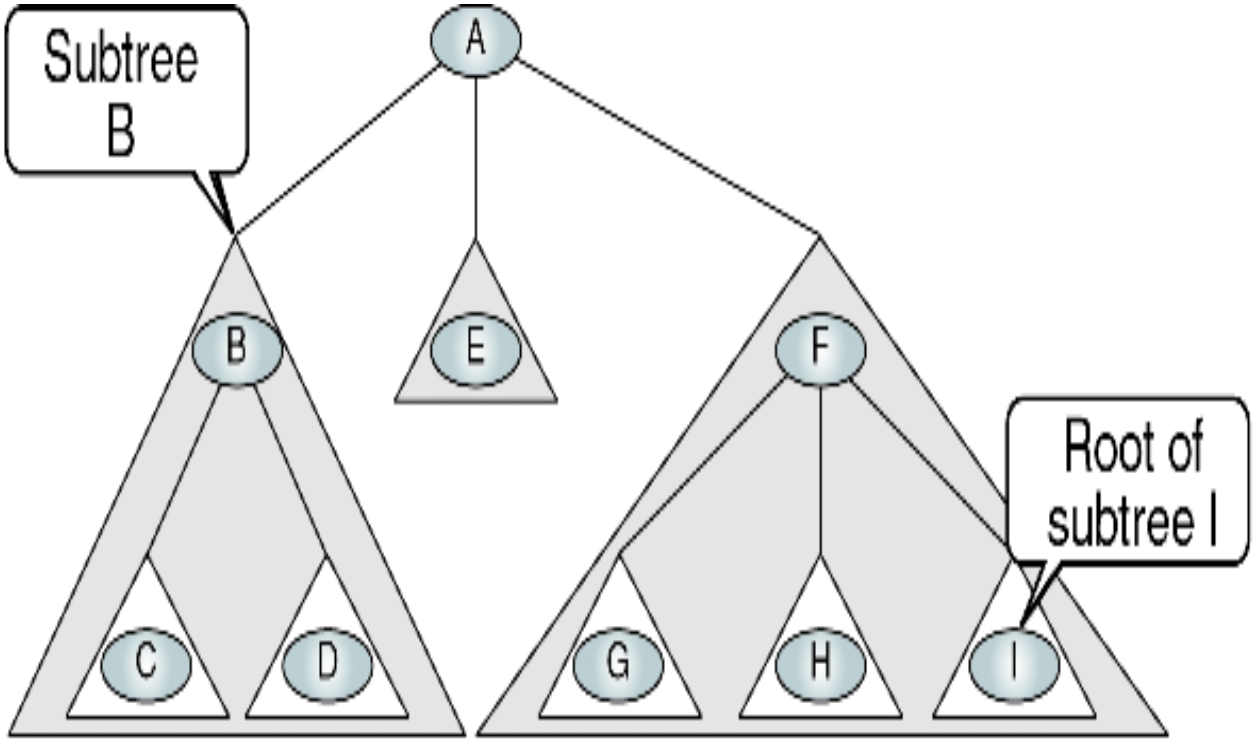


FIGURE 6-3 Subtrees

Parenthetical Listing

- Parenthetical Listing – the algebraic expression, where each open parenthesis indicates the start of a new level and each closing parenthesis completes the current level and moves up one level in the tree.

Parenthetical Listing

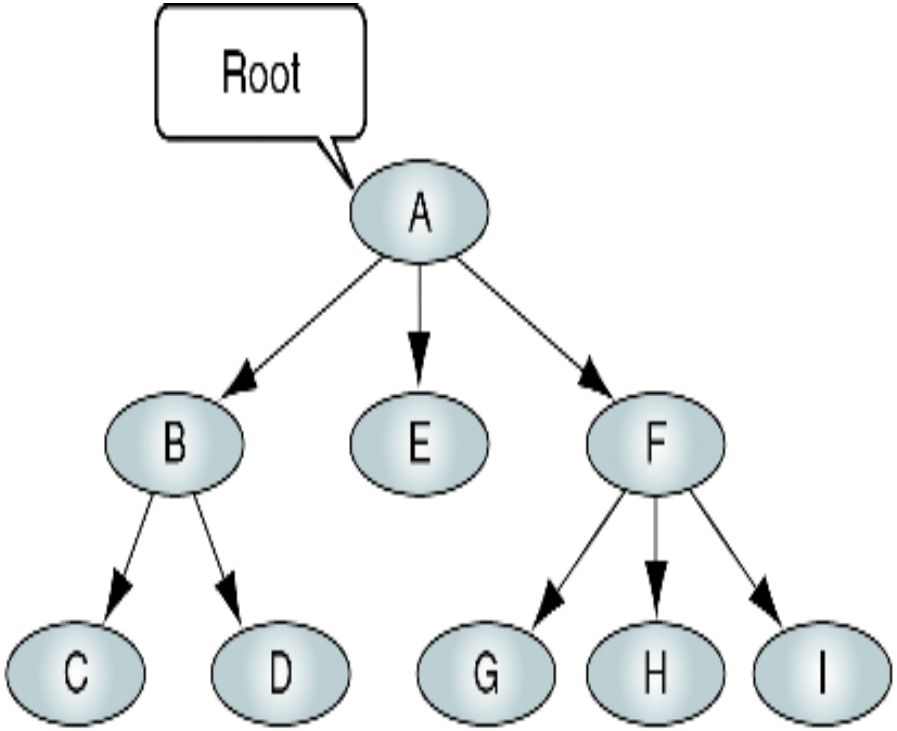


FIGURE 6-1 Tree

A (B (C D) E F (G H I))

Some Properties of Trees

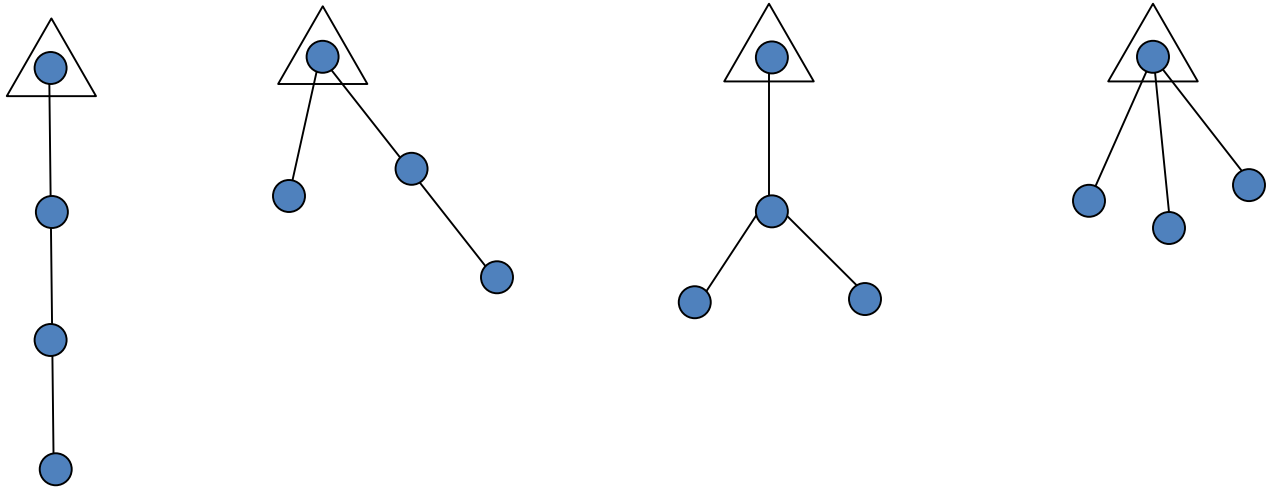
Theorem #1: There is one and only path between every pair of vertices in a tree T.

Theorem #2: A tree with n vertices has $n-1$ edges

Theorem #3: A Graph is a tree if and only if it is minimally connected.

Rooted Tree

In a tree one vertex (called root) is distinguished from all the others is called a rooted tree



Binary Trees

- A **binary tree** is a tree in which no node can have more than two subtrees; the maximum outdegree for a node is two.
- In other words, a node can have zero, one, or two subtrees.
- These subtrees are designated as the left subtree and the right subtree.

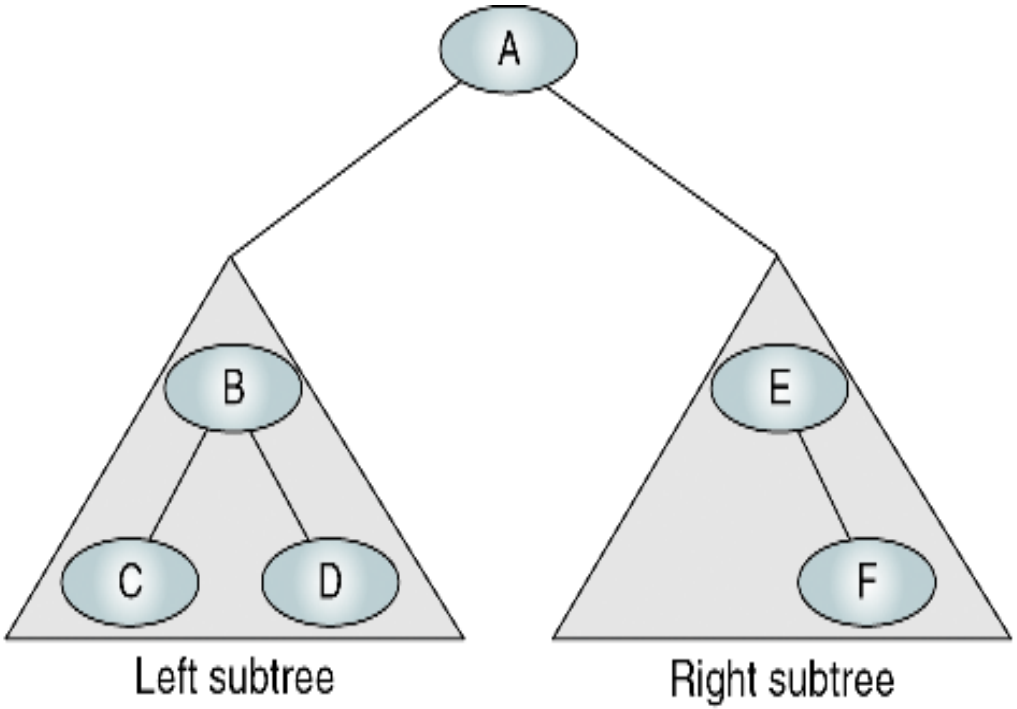


FIGURE 6-5 Binary Tree

A null tree is a tree with no nodes

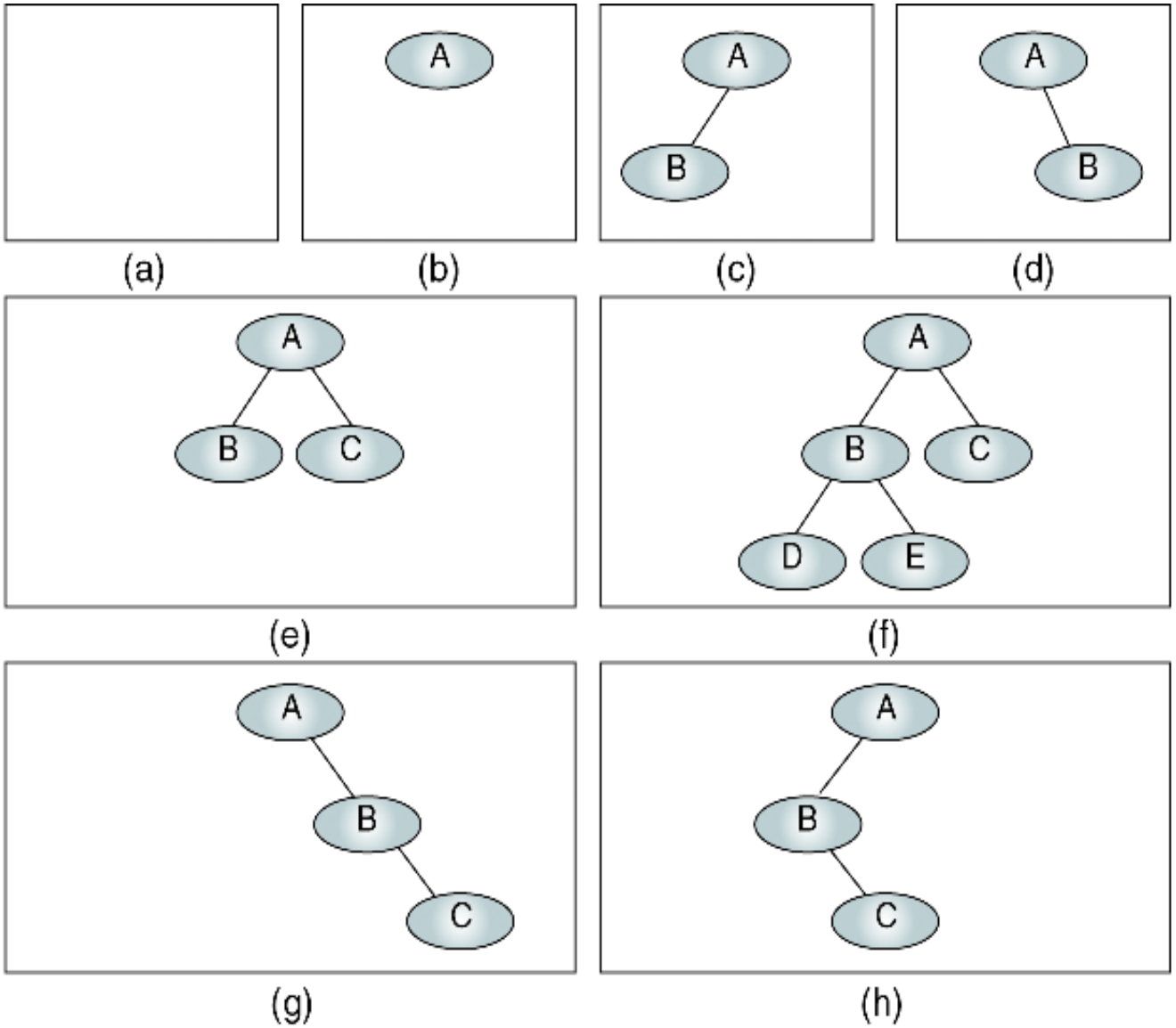


FIGURE 6-6 Collection of Binary Trees

Some Properties of Binary Trees

- The maximum number of vertices in a k-level BT is

  $2^0 + 2^1 + 2^2 + \ldots\ldots + 2^k$

- Height: The maximum level, l_{max} of any vertex in a BT is called Height of the BT. The minimum possible height is

$$minl_{max} = \lceil \log_2(n+1) - 1 \rceil$$

$$maxl_{max} = \left\lceil \frac{n-1}{2} \right\rceil$$

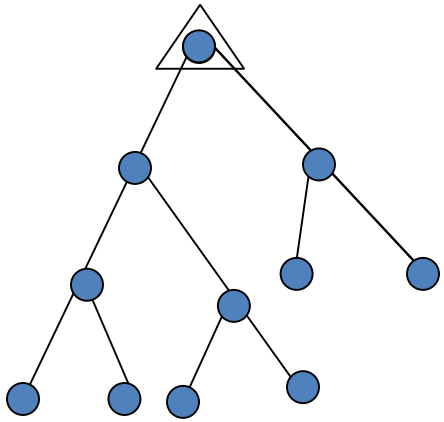
A tree with a maximum height is rare. It occurs when all of the nodes in the entire tree have only one successor.

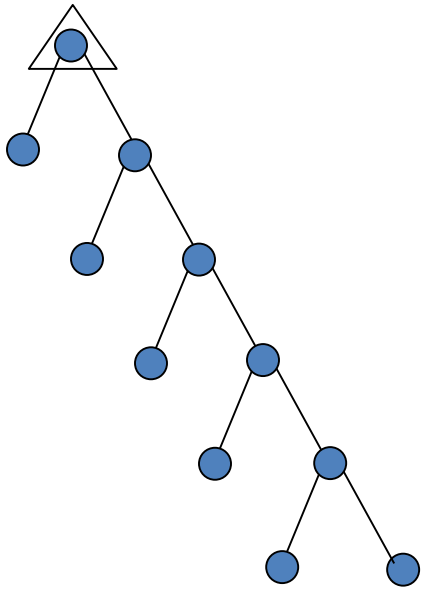
Some Properties of Binary Trees

- Example, n=11

$$\boldsymbol{minl_{max}} = \lceil \log_2(\boldsymbol{n}+\mathbf{1}) - \mathbf{1} \rceil = \mathbf{3}$$

$$\boldsymbol{maxl_{max}} = \left\lceil \frac{\boldsymbol{n}-\mathbf{1}}{\mathbf{2}} \right\rceil = \mathbf{5}$$

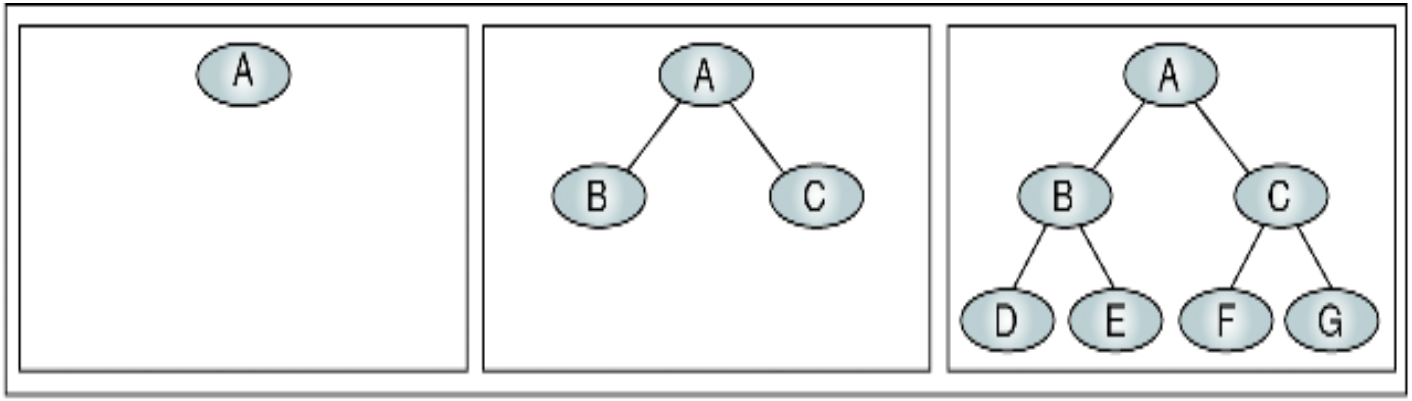




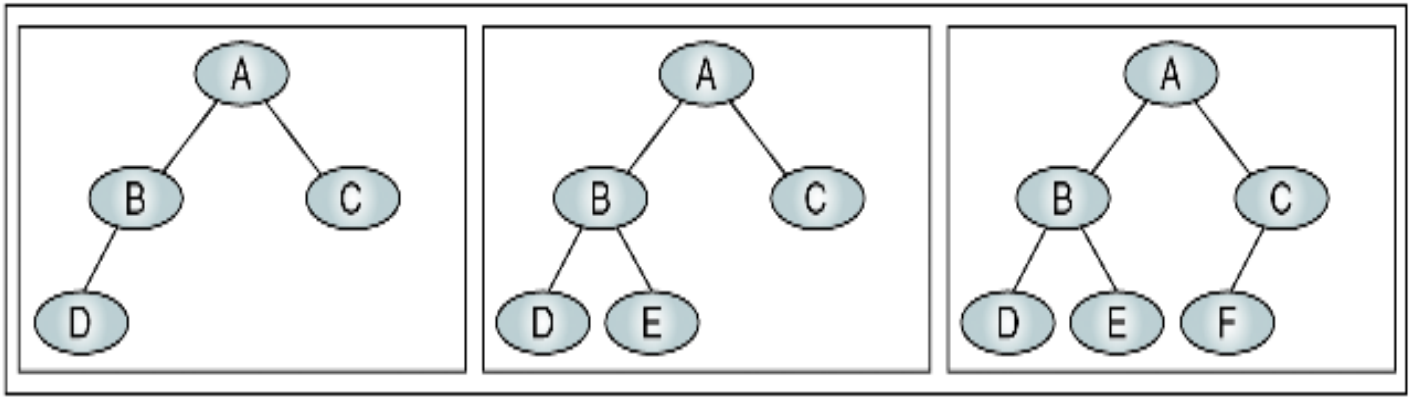
A tree with a maximum height is rare. It occurs when all of the nodes in the entire tree have only one successor.

Complete and nearly complete binary trees

- A complete tree has the maximum number of entries for its height. The maximum number is reached when the last level is full.
- A tree is considered nearly complete if it has the minimum height for its nodes and all nodes in the last level are found on the left



(a) Complete trees (at levels 0, 1, and 2)



(b) Nearly complete trees (at level 2)

FIGURE 6-7 Complete and Nearly Complete Trees

Binary Tree Traversal

A binary tree traversal requires that each node of the tree be processed once and only once in a predetermined sequence.

- Preorder Traversal (PLR)
- Inorder Traversal (LPR)
- Postorder Traversal (LRP)

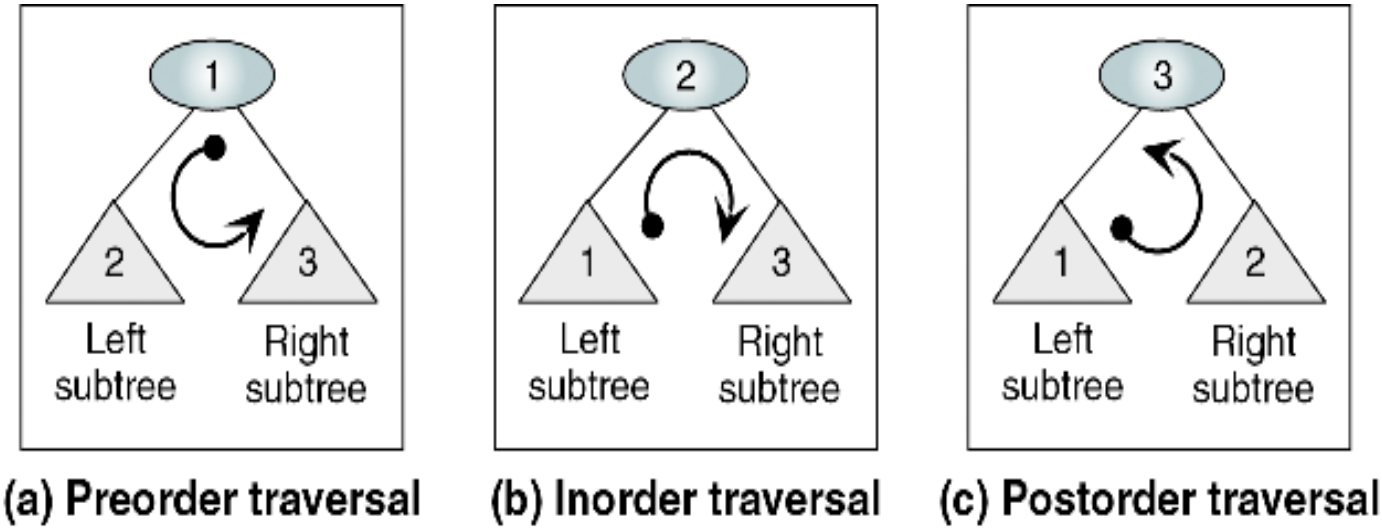


FIGURE 6-8 Binary Tree Traversals

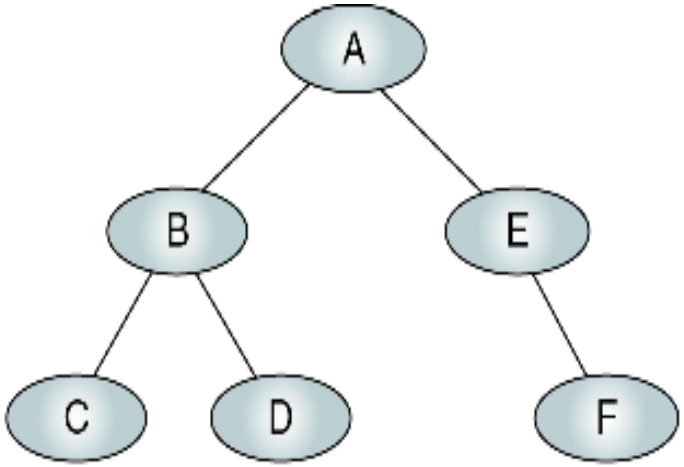


FIGURE 6-9 Binary Tree for Traversals

ALGORITHM 6-2 Preorder Traversal of a Binary Tree

```
Algorithm preOrder (root)
Traverse a binary tree in node-left-right sequence.
   Pre  root is the entry node of a tree or subtree
   Post each node has been processed in order
1 if (root is not null)
   1  process (root)
   2  preOrder (leftSubtree)
   3  preOrder (rightSubtree)
2 end if
end preOrder
```

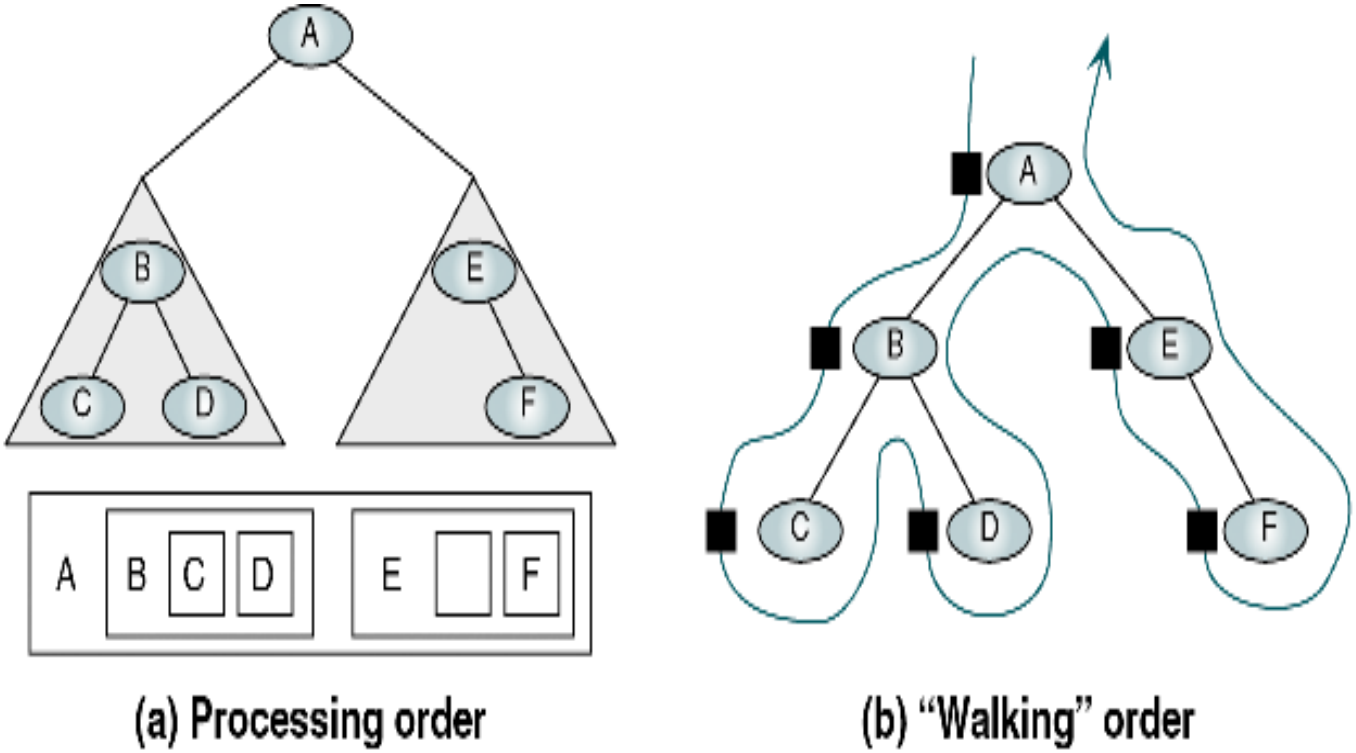


FIGURE 6-10 Preorder Traversal—A B C D E F

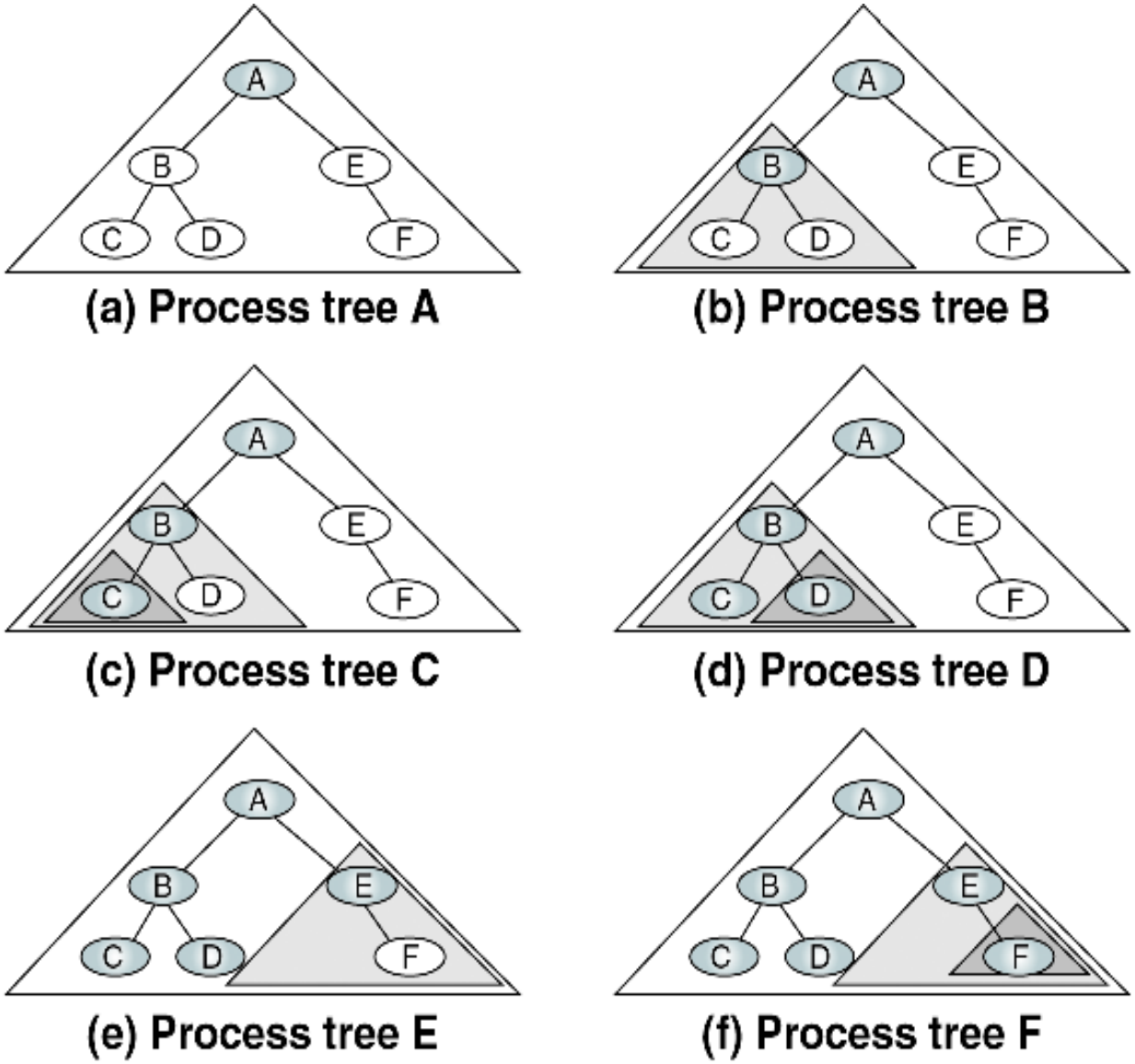


FIGURE 6-11 Algorithmic Traversal of Binary Tree

ALGORITHM 6-3 Inorder Traversal of a Binary Tree

```
Algorithm inOrder (root)
Traverse a binary tree in left-node-right sequence.
   Pre  root is the entry node of a tree or subtree
   Post each node has been processed in order
1 if (root is not null)
   1  inOrder (leftSubTree)
   2  process (root)
   3  inOrder (rightSubTree)
2 end if
end inOrder
```

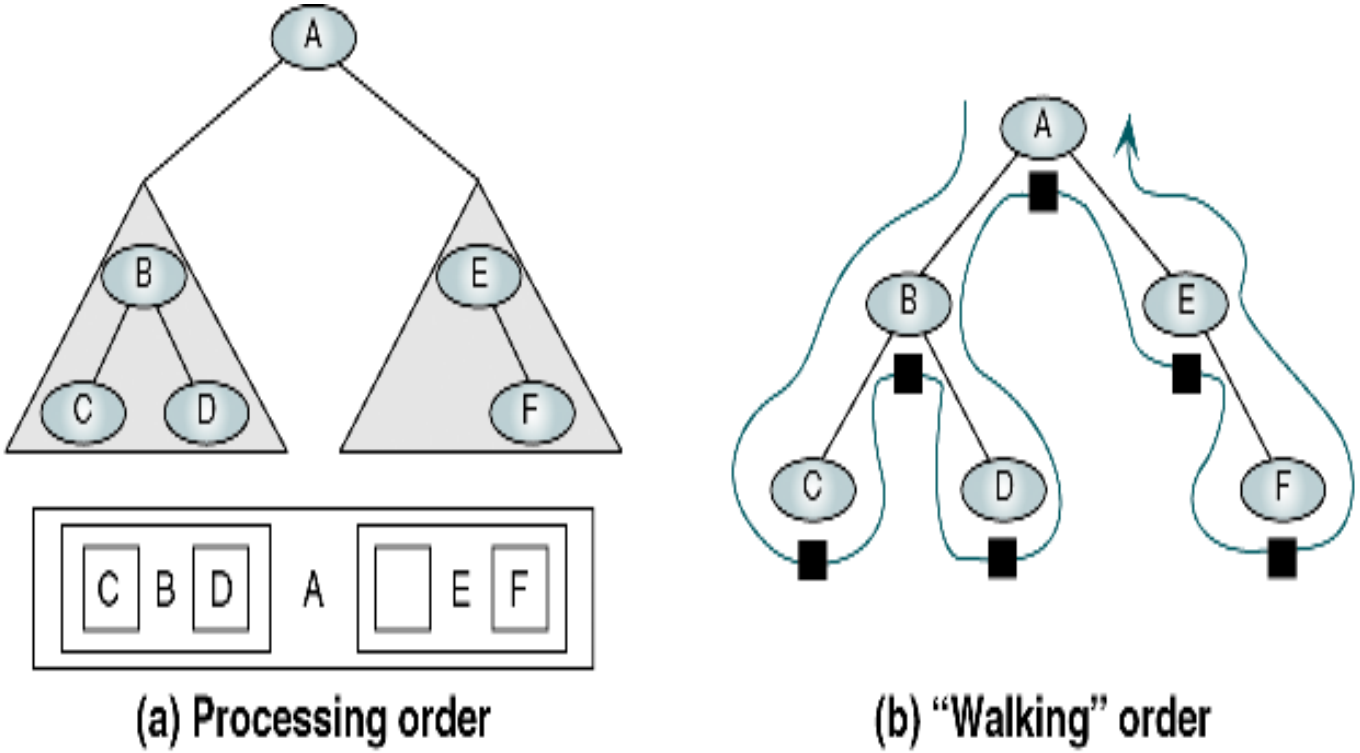


FIGURE 6-12 Inorder Traversal—C B D A E F

ALGORITHM 6-4 Postorder Traversal of a Binary Tree

```
Algorithm postOrder (root)
Traverse a binary tree in left-right-node sequence.
   Pre  root is the entry node of a tree or subtree
   Post each node has been processed in order
1 if (root is not null)
   1  postOrder (left subtree)
   2  postOrder (right subtree)
   3  process (root)
2 end if
end postOrder
```

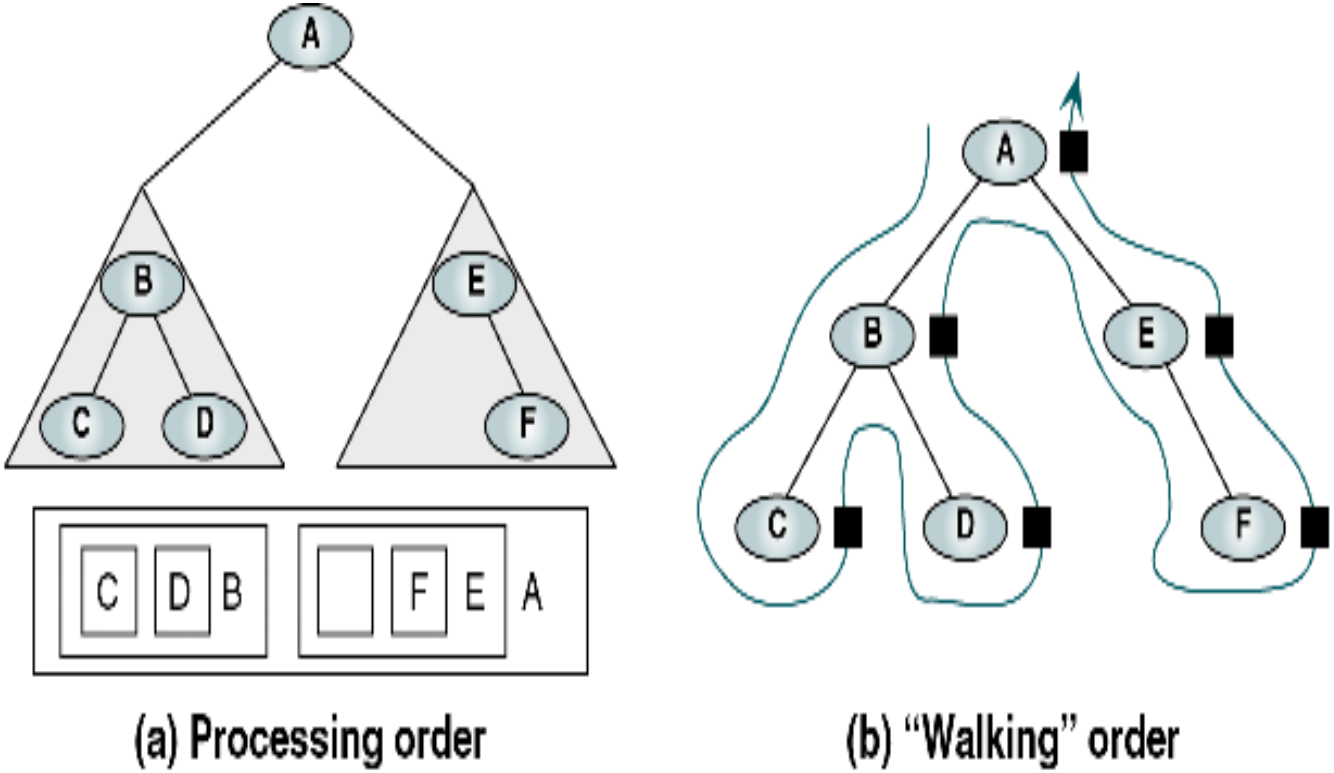


FIGURE 6-13 Postorder Traversal—C D B F E A

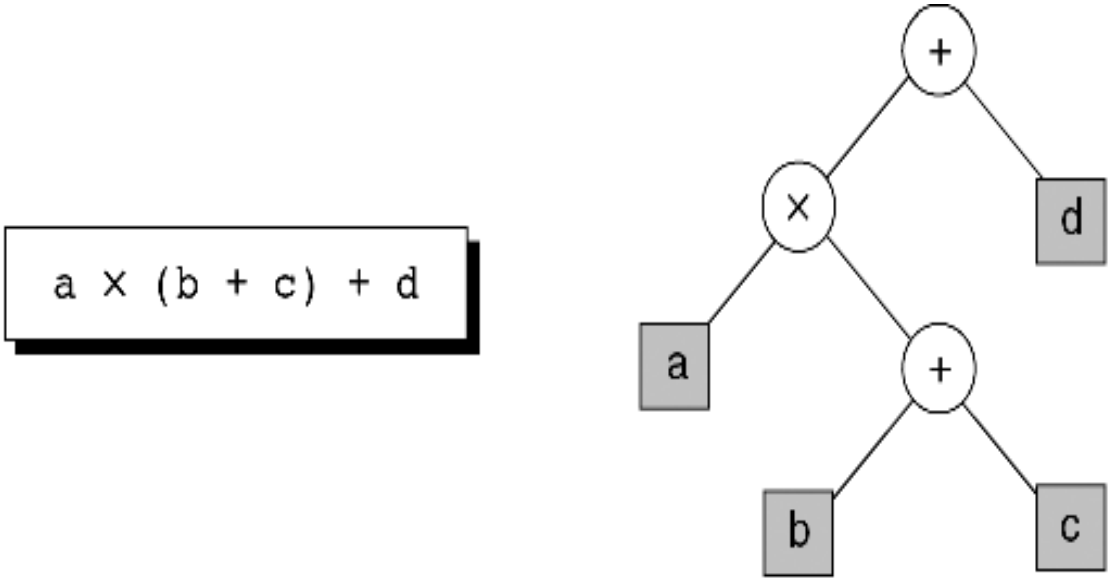


FIGURE 6-15 Infix Expression and Its Expression Tree

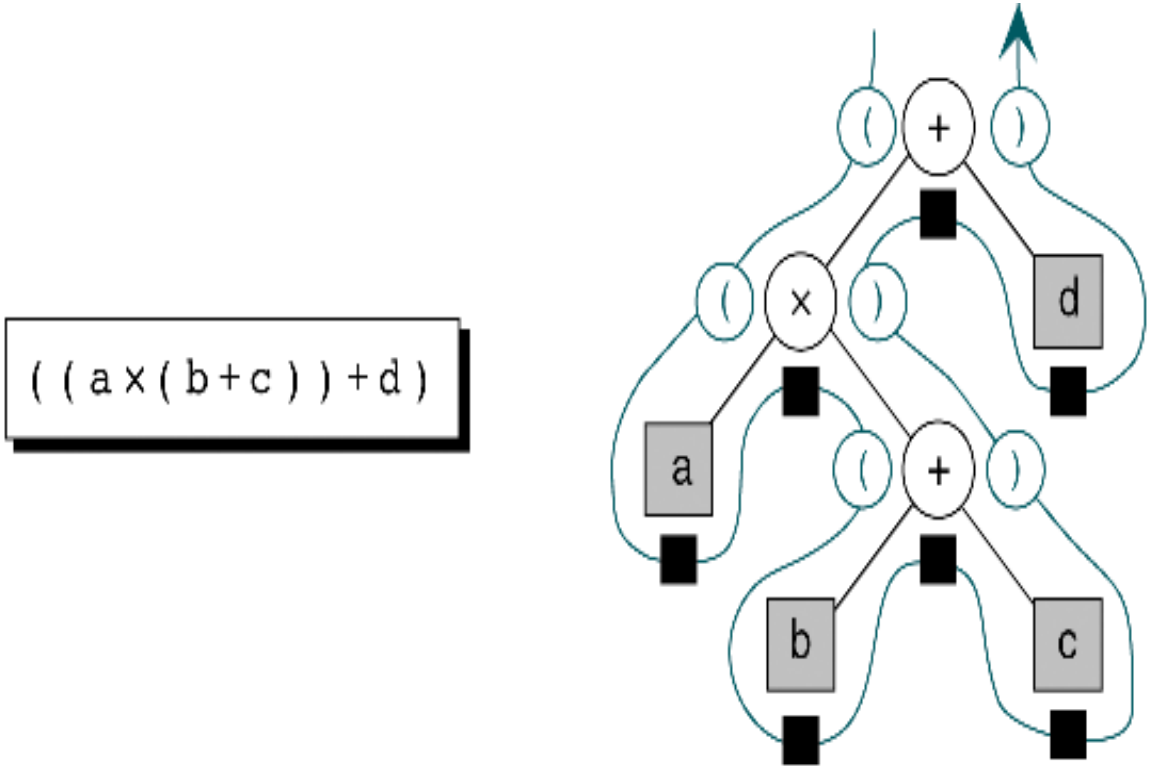


FIGURE 6-16 Infix Traversal of an Expression Tree

ALGORITHM 6-6 Infix Expression Tree Traversal

```
Algorithm infix (tree)
Print the infix expression for an expression tree.
   Pre  tree is a pointer to an expression tree
   Post the infix expression has been printed
1 if (tree not empty)
   1  if (tree token is an operand)
      1  print (tree-token)
   2  else
      1  print (open parenthesis)
      2  infix (tree left subtree)
      3  print (tree token)
      4  infix (tree right subtree)
      5  print (close parenthesis)
   3  end if
2 end if
end infix
```

ALGORITHM 6-7 Postfix Traversal of an Expression Tree

```
Algorithm postfix (tree)
Print the postfix expression for an expression tree.
   Pre  tree is a pointer to an expression tree
   Post the postfix expression has been printed
1 if (tree not empty)
```

continued

ALGORITHM 6-7 Postfix Traversal of an Expression Tree *(continued)*

```
   1  postfix (tree left subtree)
   2  postfix (tree right subtree)
   3  print   (tree token)
 2 end if
end postfix
```

ALGORITHM 6-8 Prefix Traversal of an Expression Tree

```
Algorithm prefix (tree)
Print the prefix expression for an expression tree.
   Pre  tree is a pointer to an expression tree
   Post the prefix expression has been printed
1 if (tree not empty)
   1  print  (tree token)
   2  prefix (tree left subtree)
   3  prefix (tree right subtree)
2 end if
end prefix
```

Spanning Tree

A sub graph *T of a connected graph G is called a spanning tree of G if T is a tree and T includes all the* vertices of *G*.

Minimum Spanning Tree (MST)

Let G=(V,E) be an undirected connected graph.

A sub graph T=(V,E') of G is a spanning tree of G if T is a tree.

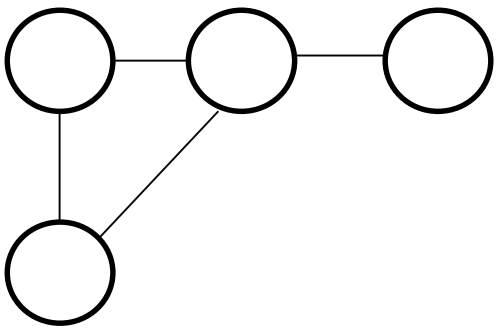
- **It is a tree (i.e., it is acyclic)**
- **It covers all the vertices *V***
  - **A single graph can have many different spanning trees.**

Minimum Spanning Tree (MST)

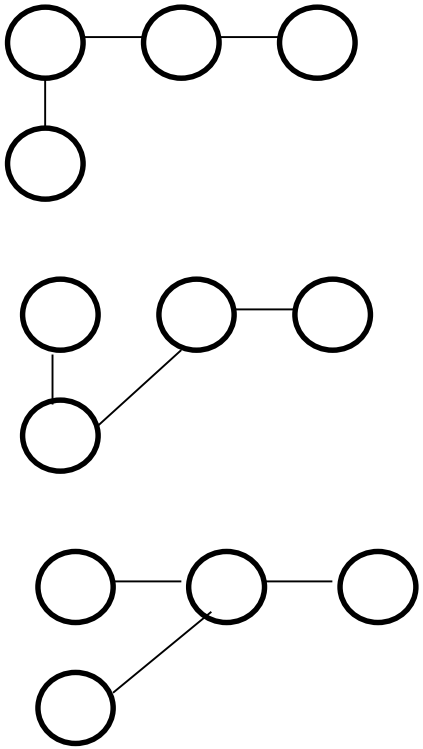
A minimum cost spanning tree is a spanning tree which has a minimum total cost.

A minimum spanning tree (MST) or minimum weight spanning tree is then a spanning tree with weight less than or equal to the weight of every other spanning tree.

Connected undirected graph

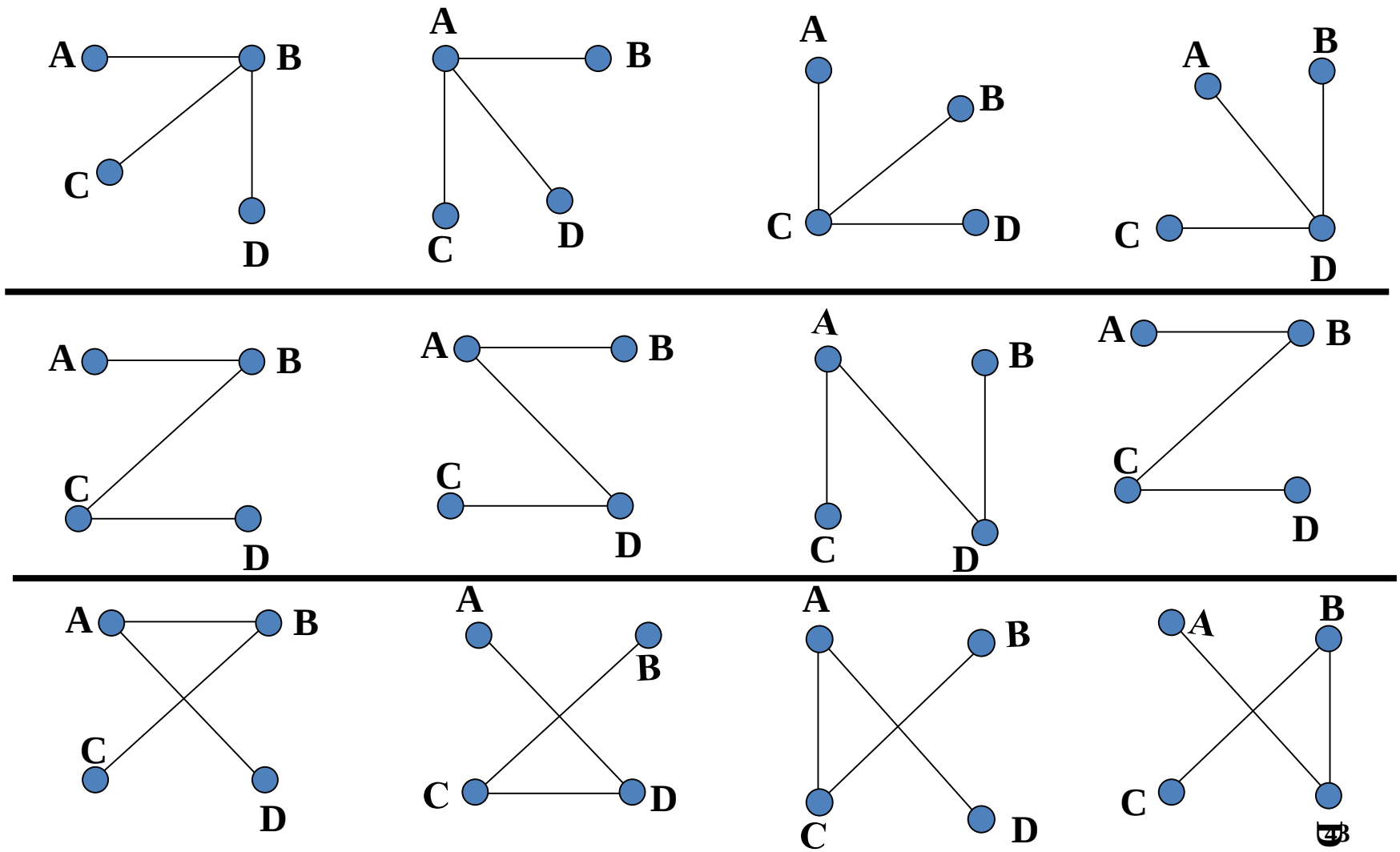


Spanning trees



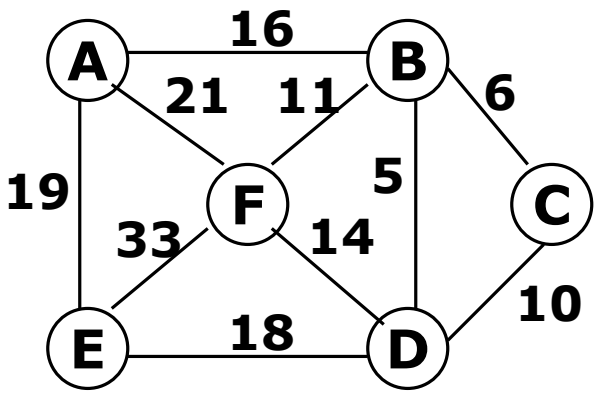
Spanning Trees

Q: How many different trees with n (n=4) distinct vertices?

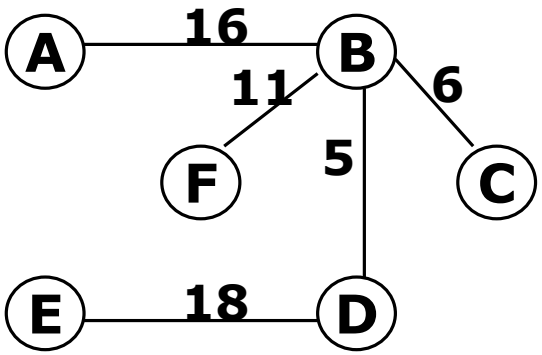


Minimum-cost spanning trees

- If we have a connected undirected graph with a **weight** (or **cost**) associated with each edge
- The cost of a spanning tree would be the sum of the costs of its edges
- A **minimum-cost spanning tree** is a spanning tree that has the lowest cost

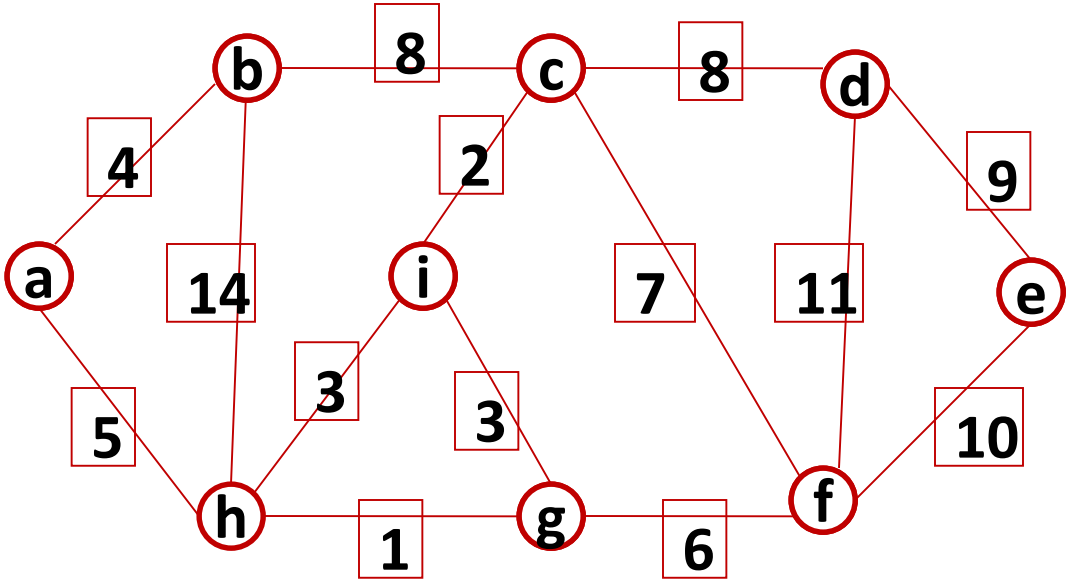


A connected, undirected graph

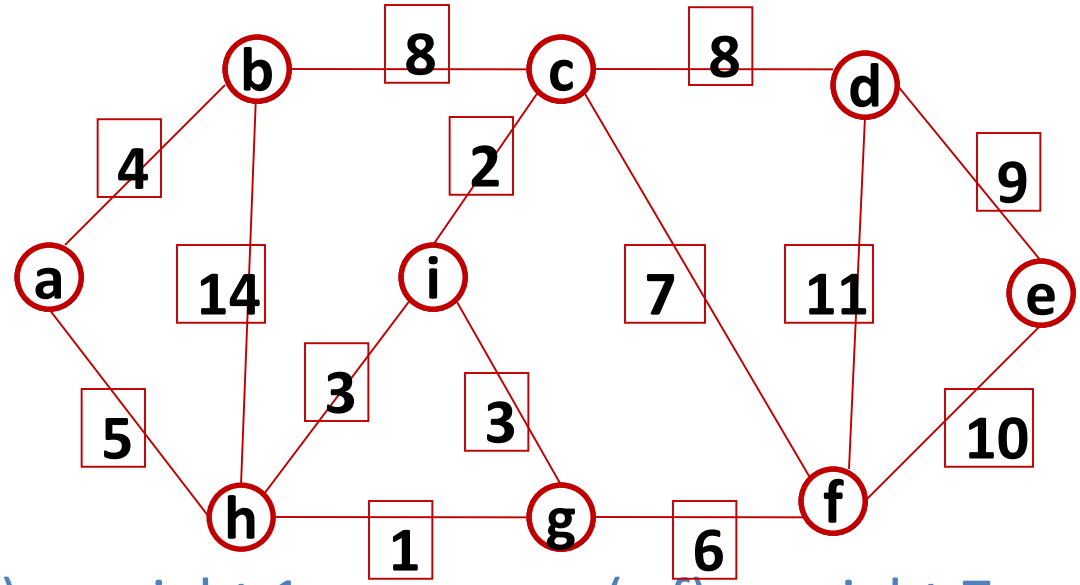


A minimum-cost spanning tree

Given: G is a connected undirected weighted graph



First sort the edges according to increasing order of their weight



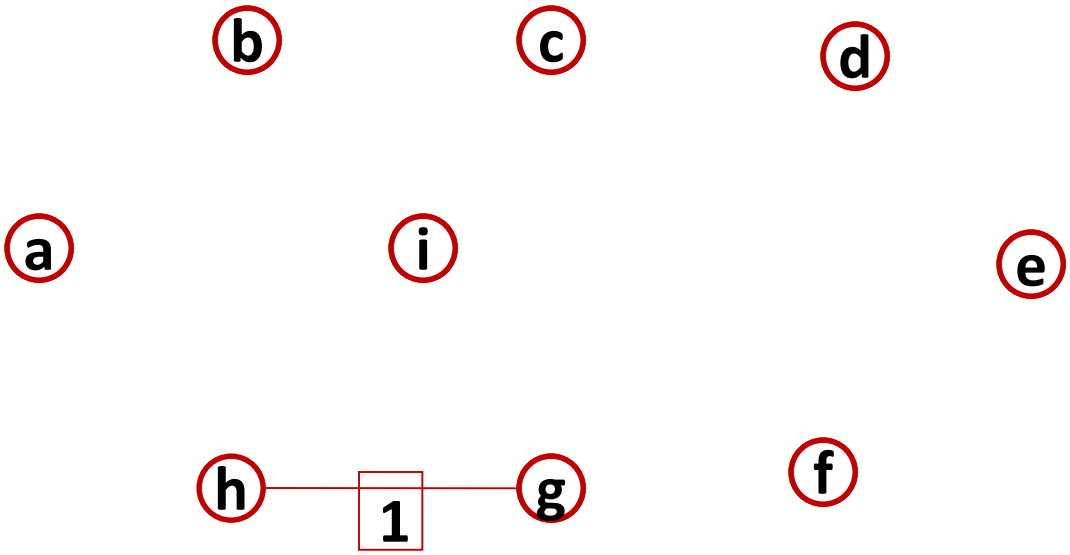
(h,g) - weight 1
(c,i) - weight 2
(i,h) - weight 3
(i,g) - weight 3
(a,b) - weight 4
(a,h) - weight 5
(g,f) - weight 6

(c,f) - weight 7
(b,c) - weight 8
(c,d) - weight 8
(d,e) - weight 9
(e,f) -weight 10
(d,f) - weight 11
(b,h) weight 14

©The McGraw-Hill Companies, Inc., 2000

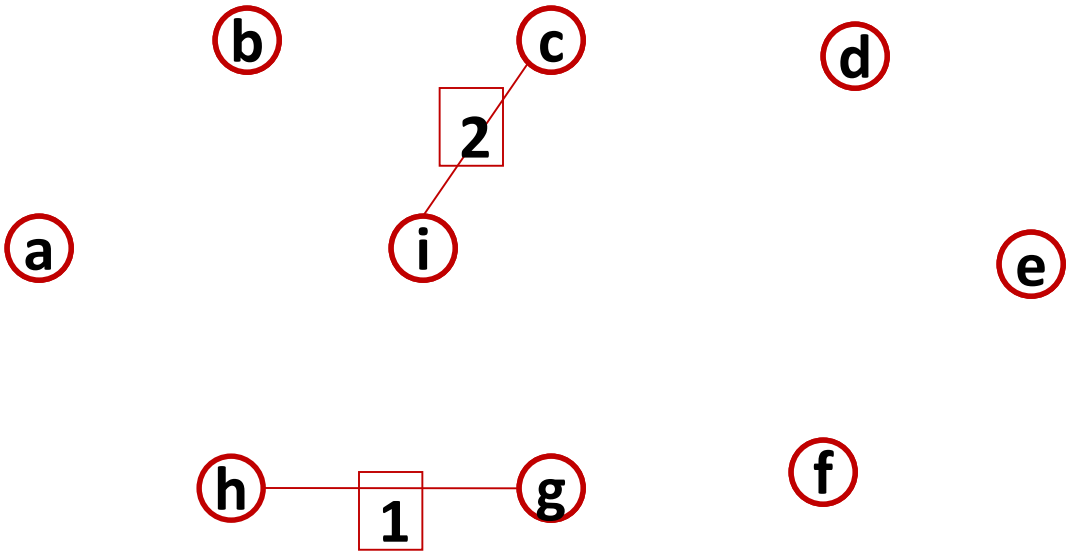
(H,G):

Select



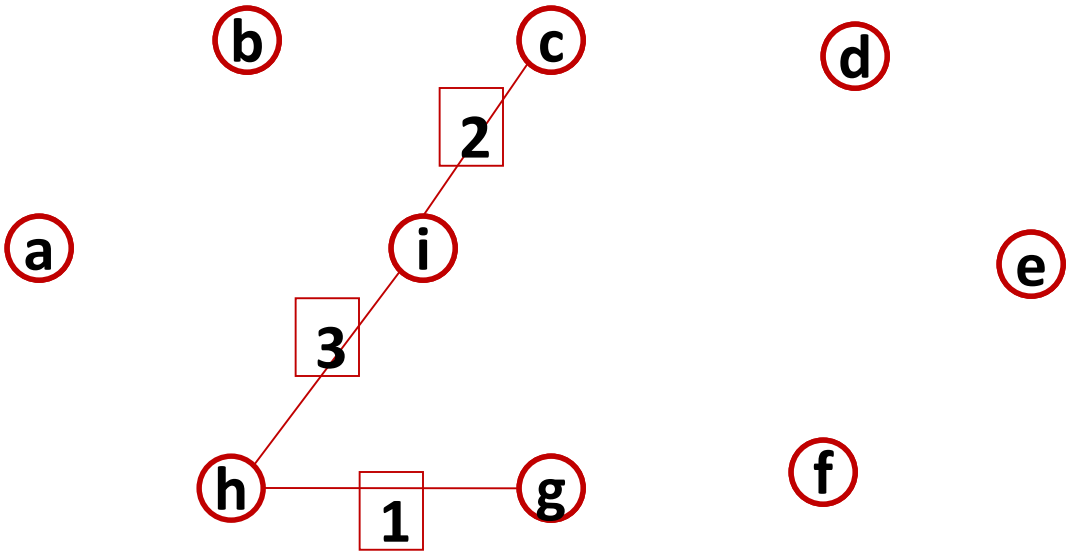
(C,I):

Select

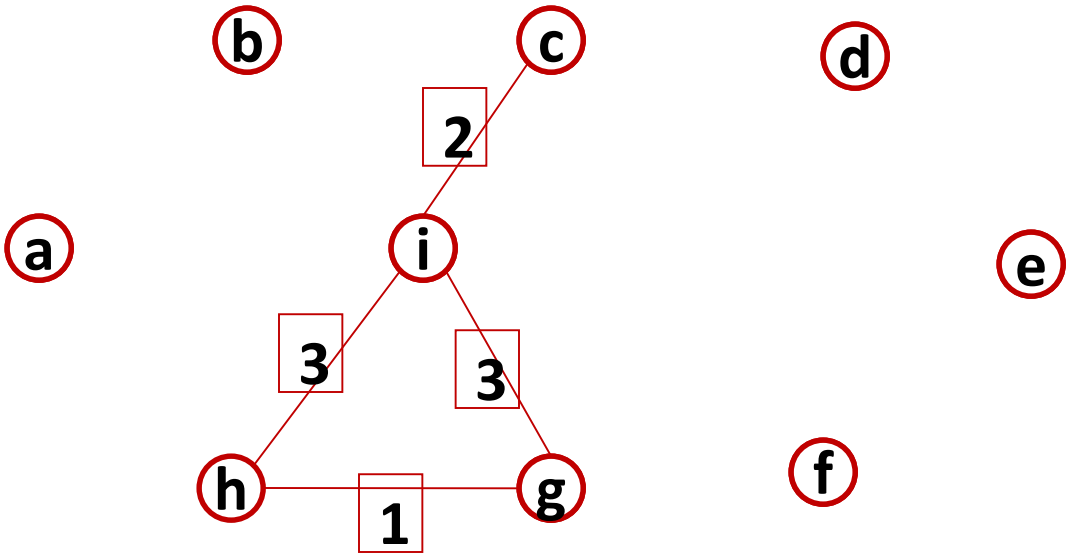


SELECT

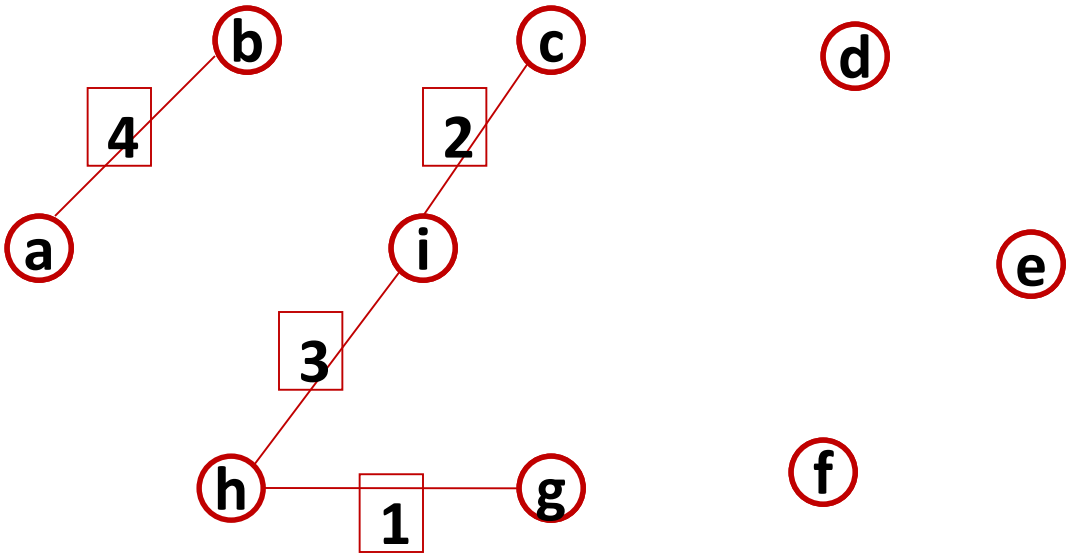
(i,h):



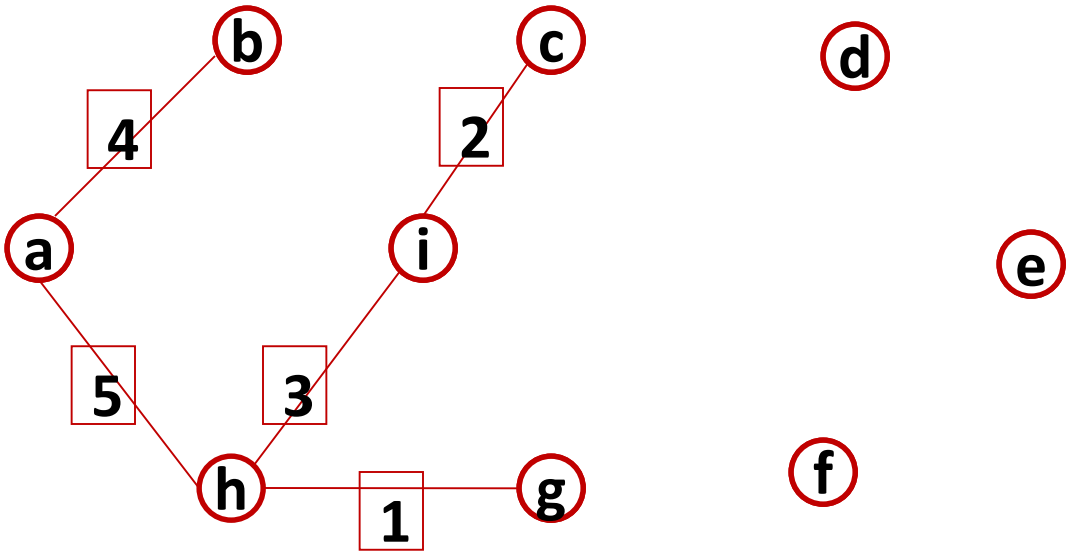
(i,g): Reject



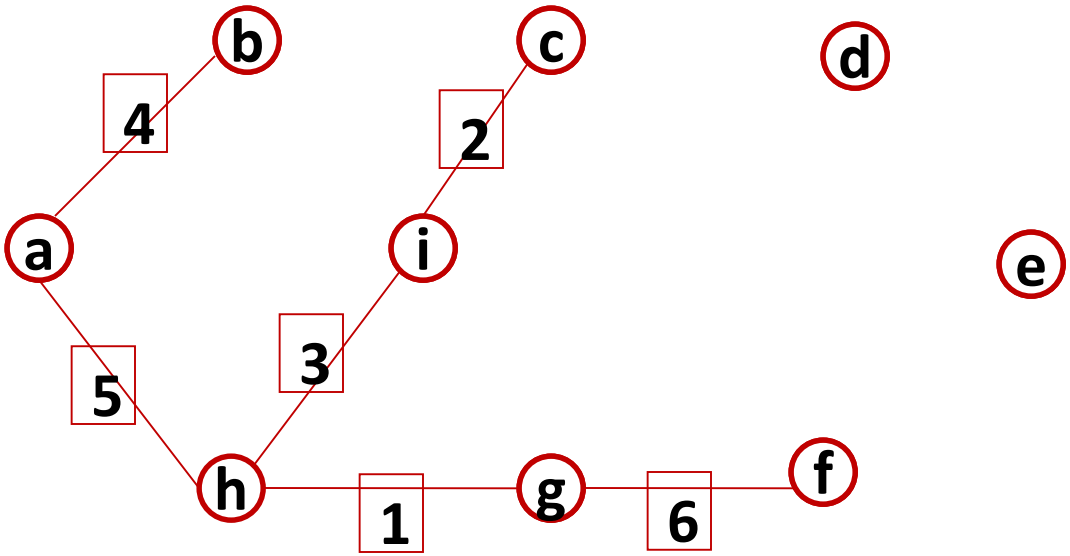
(a,b): Select



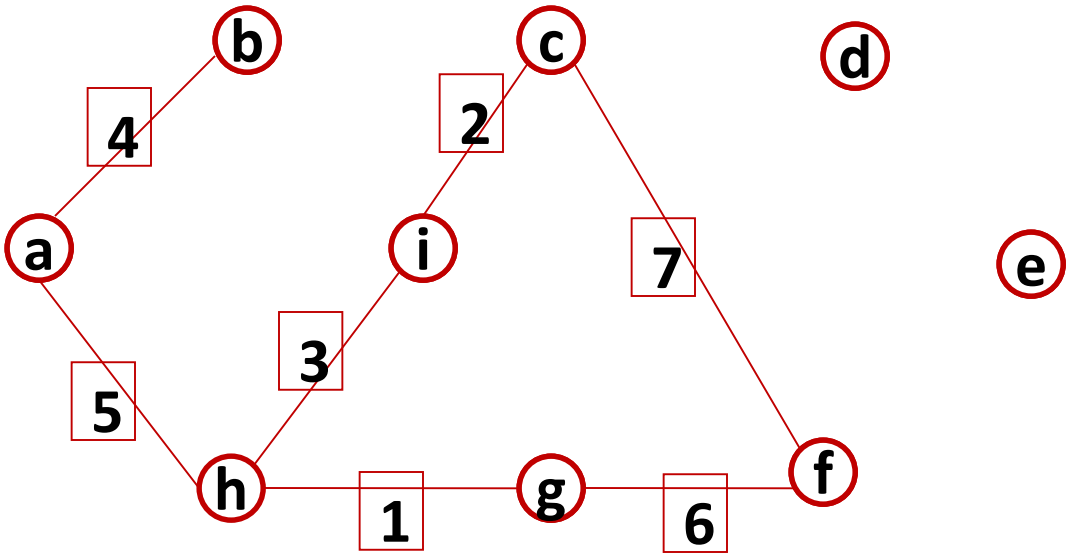
(a,h): Select



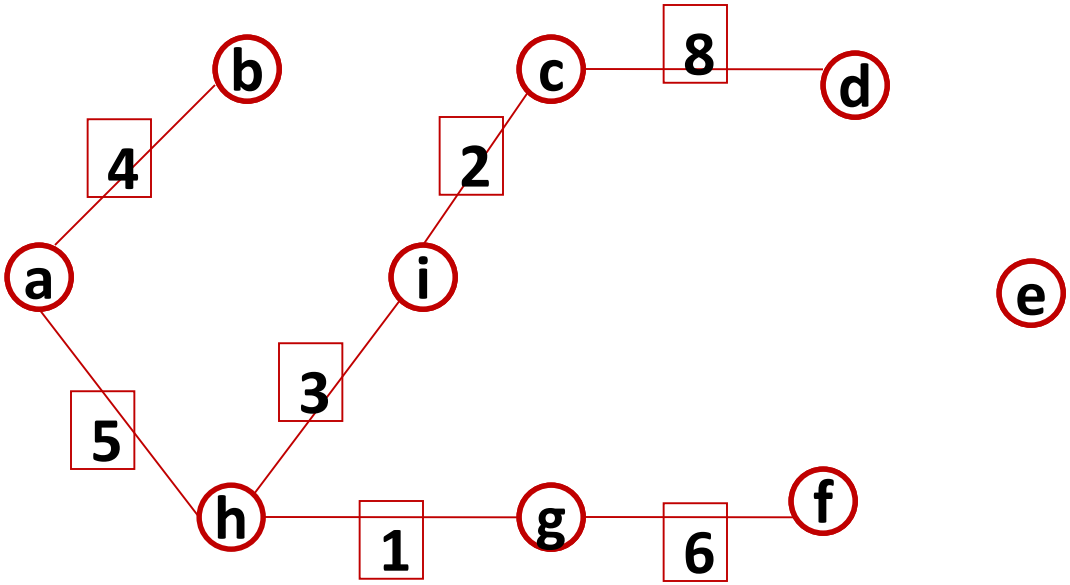
(g,f): Select



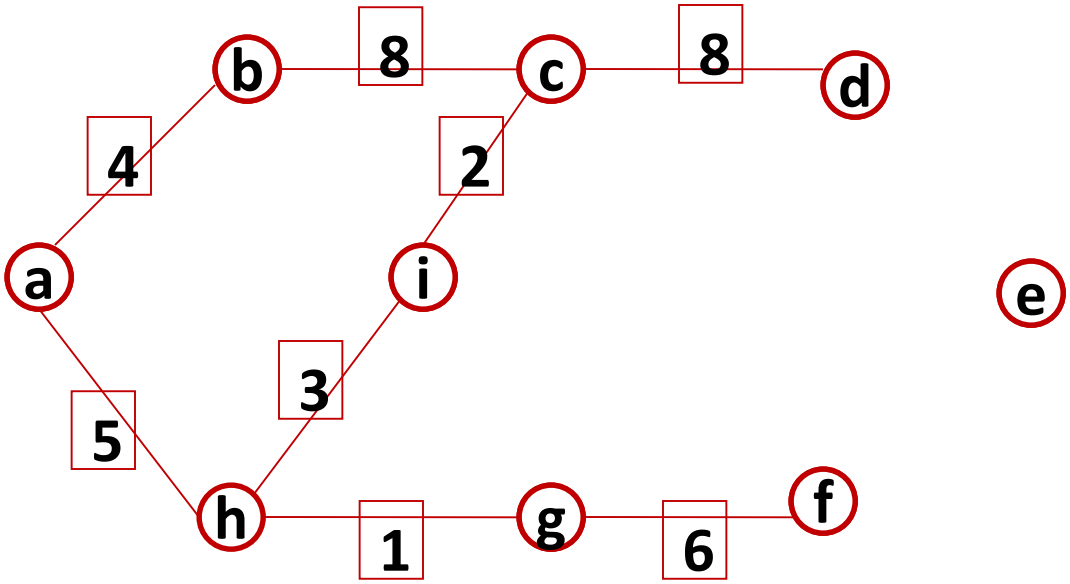
(c,f): Reject



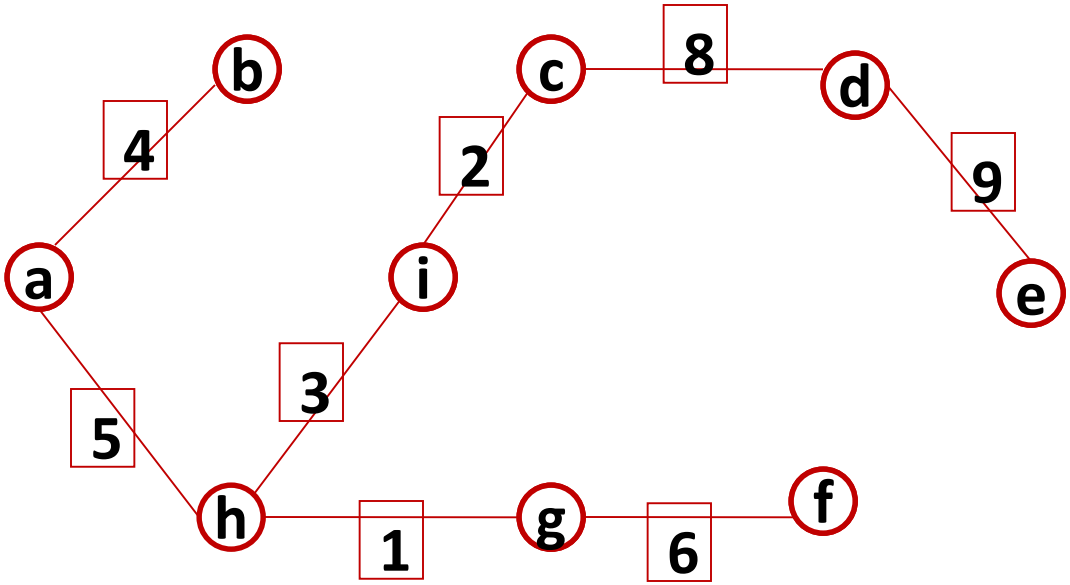
(c,d): Select



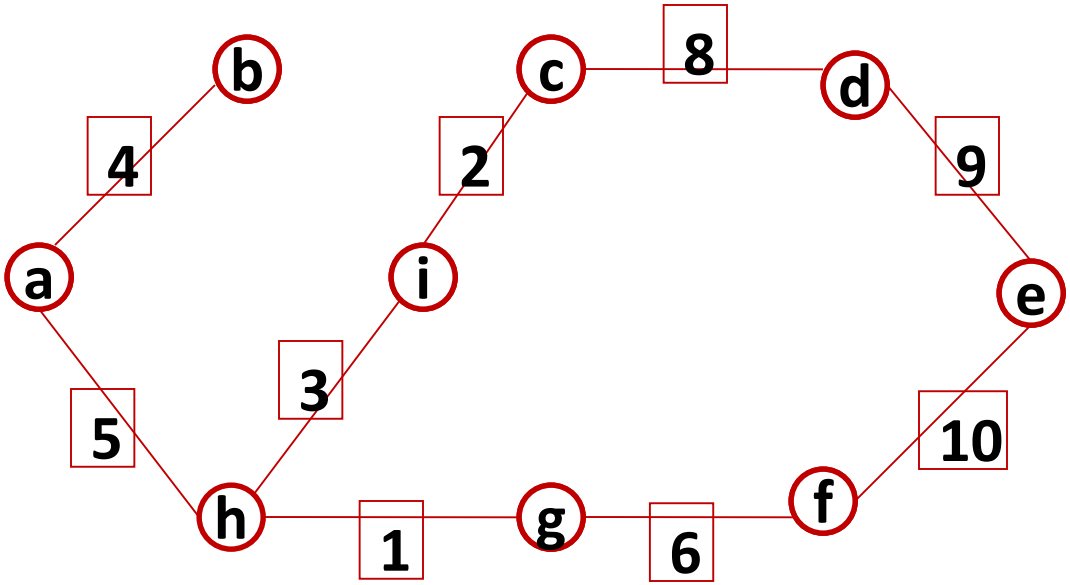
(b,c): Reject



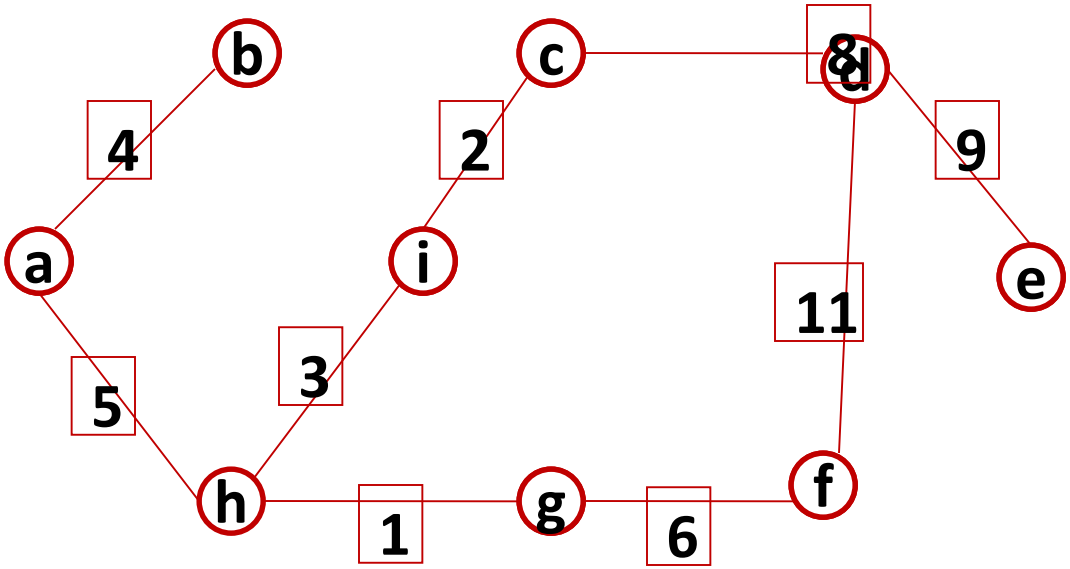
(d,e): Select



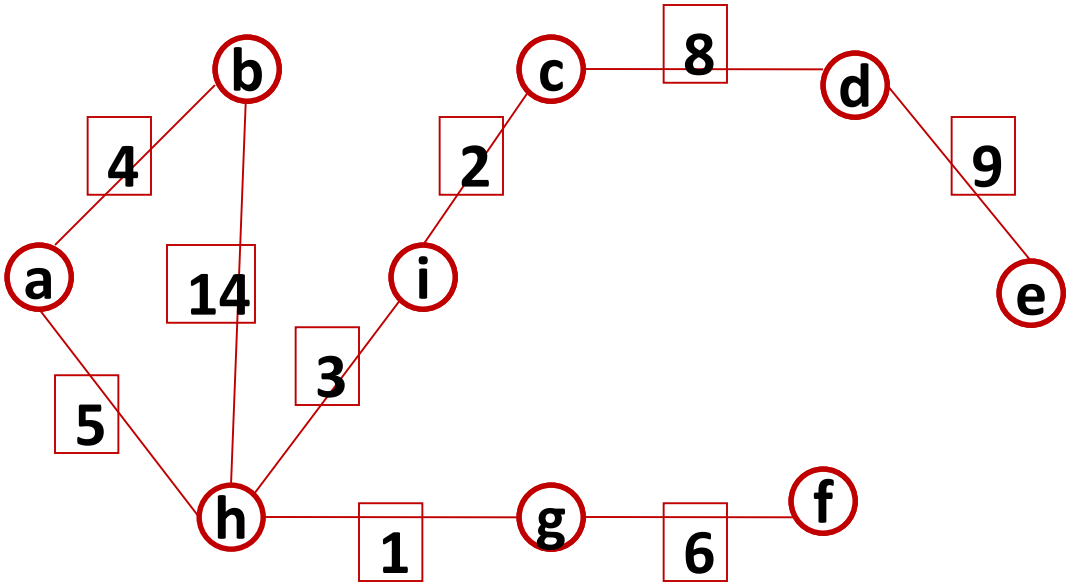
(e,f): Reject



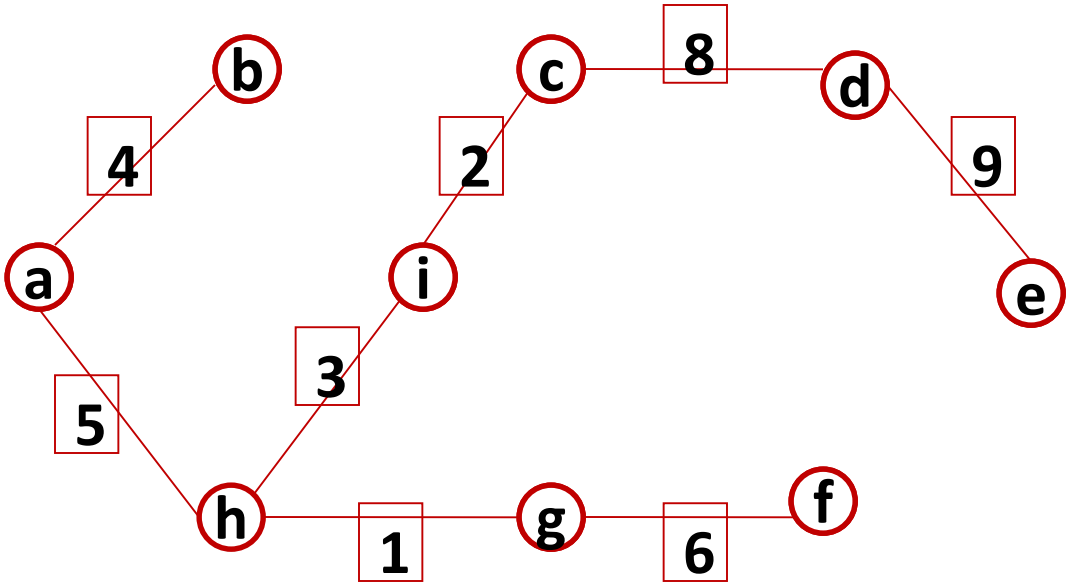
(d,f): Reject



(b,h): Reject



MINIMUM SPANNING TREE



Total weight(MST)=1+2+3+4+5+6+8+9=38

Applications of minimum spanning trees

- Consider an application where **n** stations are to be linked using a communication network.
- The laying of communication links between any two stations involves a cost.
- The problem is to obtain a network of communication links which while preserving the connectivity between stations does it with minimum cost.
- The ideal solution to the problem would be to extract a sub graph termed **minimum cost spanning tree.**
- It preserves the connectedness of the graph yields minimum cost.

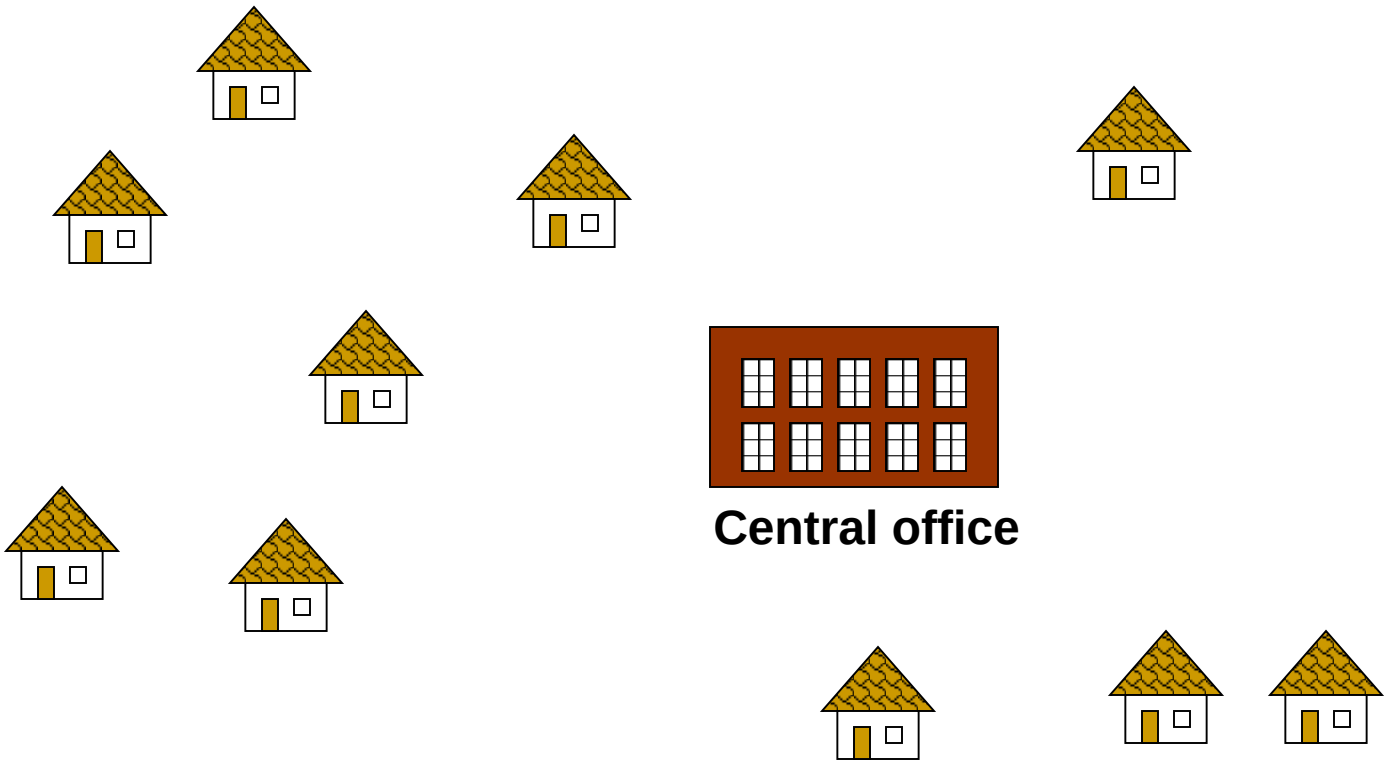
Applications cont'd

- **Suppose you want to supply a set of houses with:**
  - **electric power**
  - **water**
  - **telephone lines**
- **To keep costs down, you could connect these houses with a spanning tree (for example, power lines)**
  - **However, the houses are not all equal distances apart**
- **To reduce costs even further, you could connect the houses with a *minimum-cost* spanning tree**

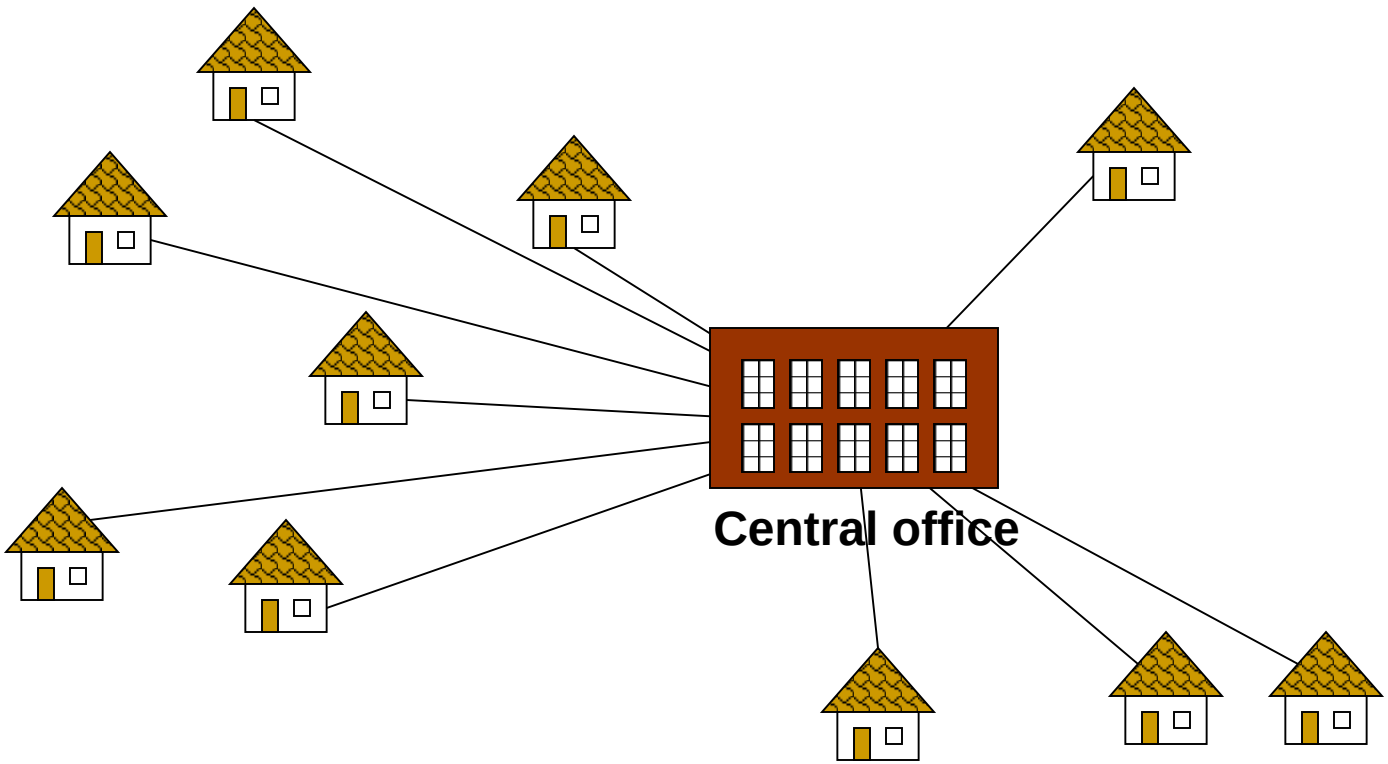
Applications cont'd

- The phone company task is to provide phone lines to a village with 10 houses, each labeled H1 through H10.
- A single cable must connects each home. The cable must run through houses H1, H2, and so forth, up through H10.
- Each node is a house, and the edges are the means by which one house can be wired up to another.
- Their task is to wire up all ten houses using the least amount of telephone wiring possible.

Problem: Laying Telephone Wire

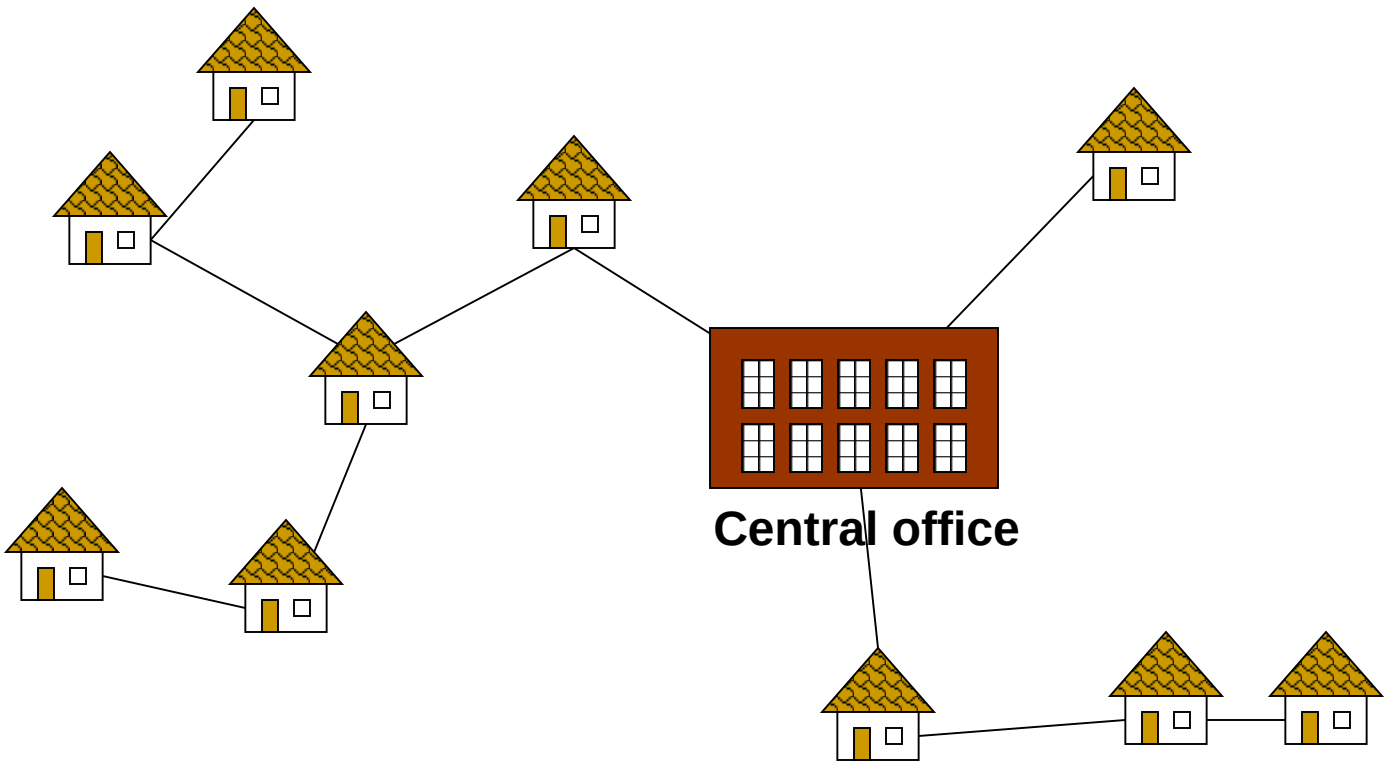


Wiring: Naïve Approach



Expensive!

Wiring: Better Approach



Minimize the total length of wire connecting the customers